Intelligence Artificielle

Contrôle Principal

Corrigé

Matériel autorisé : aucun
Durée : 90 minutes
Total de points : 14

Consignes :

- Merci de rendre la donnée à la fin du CP.
- Utiliser un stylo de couleur **bleue** ou noire (pas de crayon à papier).
- Répondre directement dans les cadres prévus ; utiliser le verso si nécessaire.
- Justifier toutes les réponses, sauf pour les QCM.
- Les téléphones portables doivent être éteints et rangés.

Exercice 1 (4 pts)

A quel type d'apprentissage correspond chacune des situations suivantes ?

1. Prédire le prix d'un appartement à partir de sa surface et de sa localisation, en disposant d'un historique de ventes

   ⇒ **Apprentissage supervisé** (régression : la cible est une valeur continue connue pour chaque exemple)

2. Regrouper les clients d'une boutique en ligne selon leurs habitudes d'achat, sans catégories prédéfinies

   ⇒ **Apprentissage non supervisé** (clustering, p. ex. *k-means*)

3. Entraîner un agent à jouer aux échecs en lui donnant une récompense lorsqu'il gagne une partie

   ⇒ **Apprentissage par renforcement** (l'agent cherche à maximiser la récompense cumulée)

4. Classer des e-mails en *spam* / *non-spam* à partir d'un corpus d'e-mails déjà étiquetés

   ⇒ **Apprentissage supervisé** (classification binaire)

Exercice 2 (4 pts)

Écrivez en Python (avec `scikit-learn`) une fonction `evaluer(X, y)` qui :

1. sépare les données en un jeu d'entraînement (80 %) et un jeu de test (20 %),
2. normalise les attributs (moyenne nulle, écart-type unitaire) en s'ajustant **uniquement** sur le jeu d'entraînement,
3. entraîne un classifieur des k plus proches voisins avec $k = 5$,
4. et retourne l'exactitude (*accuracy*) obtenue sur le jeu de test.

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

def evaluer(X, y):
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)
    scaler = StandardScaler().fit(X_tr)   # fit sur l'entraînement seulement
    X_tr, X_te = scaler.transform(X_tr), scaler.transform(X_te)
    knn = KNeighborsClassifier(n_neighbors=5).fit(X_tr, y_tr)
    return knn.score(X_te, y_te)
```

Exercice 3 — QCM (3 pts)

Pour chaque question, cochez **la ou les** réponse(s) correcte(s) (1 pt par question).

1. Quel est le rôle du jeu de **validation** ?

   - ☐ Entraîner les poids du modèle
   - ☒ Choisir les hyperparamètres du modèle
   - ☐ Mesurer la performance finale publiée

2. Quelles méthodes sont des méthodes d'**ensemble** ? *(plusieurs réponses possibles)*

   - ☒ *Random forest*
   - ☒ *Gradient boosting*
   - ☐ *k-means*
   - ☐ ACP (PCA)

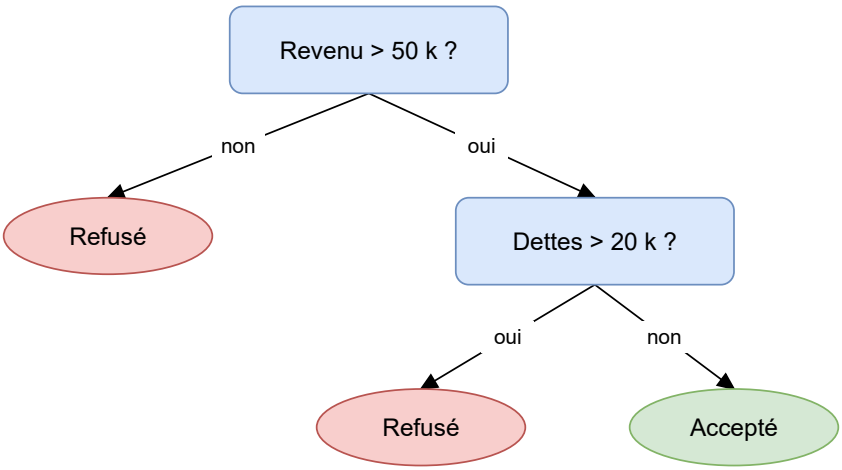
3. Un classifieur obtient 40 vrais positifs, 10 faux positifs et 20 faux négatifs. Sa **précision** vaut :

   ☐ 0.67 ☒ 0.80 ☐ 0.50 ☐ 0.60

   Précision $= \dfrac{VP}{VP+FP} = \dfrac{40}{40+10} = 0.80$ (le rappel vaut $\dfrac{40}{60} \approx 0.67$).

Exercice 4 (3 pts)

Une banque utilise l'arbre de décision suivant pour accepter ou refuser une demande de crédit.



1. Quelle décision l'arbre prend-il pour un client ayant un revenu de 65 k et 10 k de dettes ?

   ⇒ **Accepté** (revenu > 50 k : oui, puis dettes > 20 k : non)

2. Quelle décision l'arbre prend-il pour un client ayant un revenu de 40 k et aucune dette ?

   ⇒ **Refusé** (revenu > 50 k : non ; les dettes ne sont pas examinées)

3. Quelle est la profondeur de cet arbre ? Citez un critère permettant de choisir l'attribut testé à la racine.

   ⇒ Profondeur **2**. Critère : **gain d'information** (entropie) ou **indice de Gini** ; on choisit l'attribut qui sépare le mieux les classes.